

INFORMATION TECHNOLOGY UNIVERSITY

Artificial intelligence
Fall 2026

Instructor	Ms. Maria Mariam
Lab Instructors	Muhammad Talha Qureshi Assadullah Farrukh Maha Faisal Hafiz Abdullah Nauman Arif Haris Alam

Lab 06: Uniform Cost Search and Greedy Best-First Search

Submission Format: YourName_RollNo_Lab05.py

Task 1 — Priority Queue (Min-Heap)

Objective

Implement a min-priority queue using a binary min-heap stored in a Python list. This queue will be reused in Task 2 to build both search algorithms.

Problem statement

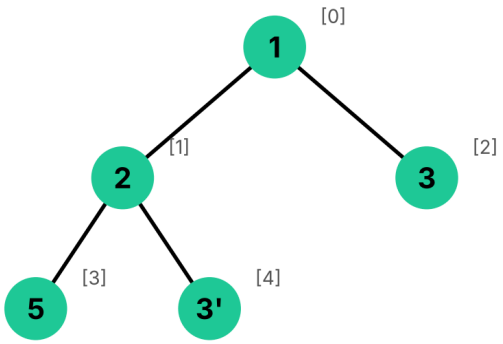
Create a class MinPriorityQueue that stores (priority, item) pairs and always removes the pair with the smallest priority first. Each operation must keep the heap property intact.

Representation

The heap is stored in a list called self.heap. For an element at index i:

- Parent index = (i - 1) // 2
- Left child index = 2*i + 1
- Right child index = 2*i + 2

The figure below shows the heap obtained after pushing the priorities 5, 1, 3, 2, 3 (the last 3 is written as 3′). The number in brackets is the list index.



Required methods

- **push(priority, item)** — insert a new pair and move it up until the heap property holds.
- **pop()** — remove and return the (priority, item) pair with the smallest priority. Move the last element to the root and sift it down.
- **peek()** — return the smallest pair without removing it.
- **is_empty()** — return True if the queue has no elements.

- **__len__()** — return the number of elements.
- **_sift_up(i)** and **_sift_down(i)** — helper methods used by push and pop.

If two pairs have the same priority, the one that was inserted first must be removed first. Hint: store (priority, counter, item) internally, where the counter increases by one with every push.

Calling pop() or peek() on an empty queue must raise IndexError("priority queue is empty").

Starter code

```
class MinPriorityQueue:
    def __init__(self):
        self.heap = []
        self.counter = 0

    def __len__(self):
        # TODO

    def is_empty(self):
        # TODO

    def push(self, priority, item):
        # TODO

    def pop(self):
        # TODO

    def peek(self):
        # TODO

    def _sift_up(self, i):
        # TODO

    def _sift_down(self, i):
        # TODO
```

Sample input

```
pq = MinPriorityQueue()
for p, x in [(5, 'e'), (1, 'a'), (3, 'c'), (2, 'b'), (3, 'c2')]:
    pq.push(p, x)

print(pq.peek())
print(len(pq))
while not pq.is_empty():
    print(pq.pop())

pq.pop()
```

Sample output

```
(1, 'a')
(2, 'b')
(3, 'c')
(3, 'c2')
(5, 'e')
IndexError: priority queue is empty
```

Note: *Do not use heapq or queue.PriorityQueue.*

Task 2a — Uniform Cost Search (UCS)

Objective

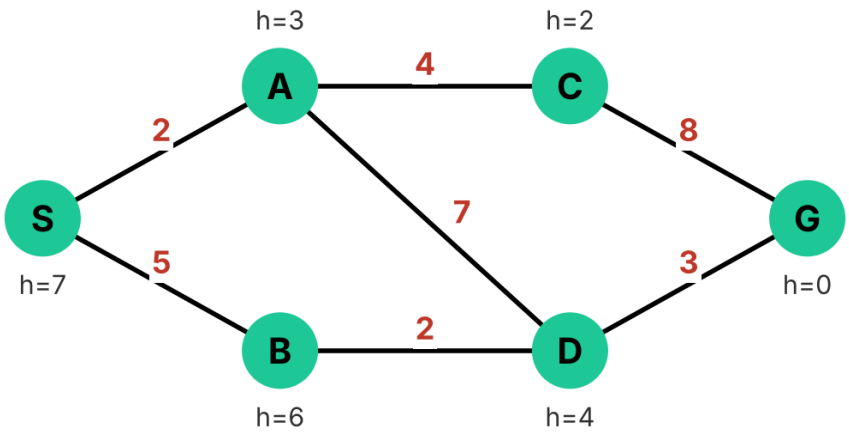
Write a UCS function that finds the cheapest path between a start node and a goal node in a weighted undirected graph. Use your MinPriorityQueue from Task 1.

Problem statement

Given a weighted graph, a start node and a goal node, always expand the node with the smallest path cost g(n) from the start. Return the path, its total cost and the order in which nodes were expanded.

Sample input

The graph below has edge costs written in red. Find a path from S to G.



The graph is stored as a dictionary. Each node maps to a list of (neighbour, cost) pairs:

```
graph = {
    'S': [('A', 2), ('B', 5)],
    'A': [('S', 2), ('C', 4), ('D', 7)],
    'B': [('S', 5), ('D', 2)],
    'C': [('A', 4), ('G', 8)],
    'D': [('A', 7), ('B', 2), ('G', 3)],
    'G': [('C', 8), ('D', 3)],
}
start = 'S'
goal = 'G'
```

Your function must have this signature:

```
def ucs(graph, start, goal):
    # return (path, total_cost, expansion_order)
```

Rules

- Use the priority g(n), the total cost from the start node to n.
- Keep an explored set. If a popped node is already explored, skip it (it is an outdated entry).
- Add a node to expansion_order when it is popped and not skipped. The goal is included.
- Remember the parent of every node so that the path can be rebuilt at the end.
- If no path exists, return (None, None, expansion_order).

Sample output

```
UCS path: ['S', 'B', 'D', 'G']
Total cost: 10
Nodes expanded: ['S', 'A', 'B', 'C', 'D', 'G']
```

Note: *Do not use heapq or queue.PriorityQueue. Use the class you wrote in Task 1.*

Task 2b — Greedy Best-First Search (GBFS)

Objective

Write a GBFS function for the same graph. Instead of the path cost, GBFS uses a heuristic h(n), an estimate of how far node n is from the goal.

Problem statement

Given the graph, a heuristic table, a start node and a goal node, always expand the node with the smallest h(n). Return the path found, its real total cost and the order in which nodes were expanded.

Sample input

Use the same graph as in Task 2a, together with these heuristic values (they are also shown under each node in the figure):

Node	S	A	B	C	D	G
h(n)	7	3	6	2	4	0

```
h = {'S': 7, 'A': 3, 'B': 6, 'C': 2, 'D': 4, 'G': 0}
```

Your function must have this signature:

```
def gbfs(graph, h, start, goal):
    # return (path, total_cost, expansion_order)
```

Rules

- Use the priority h(n) of the node.
- Keep an explored set and skip nodes that are already explored.
- Only push neighbours that are not yet explored.
- The returned cost must be the real sum of edge costs along the path, not the h value.
- If no path exists, return (None, None, expansion_order).

Sample output

```
GBFS path: ['S', 'A', 'C', 'G']
Total cost: 14
Nodes expanded: ['S', 'A', 'C', 'G']
```

Note: *GBFS only looks at the estimate h(n), so the path it returns is not always the cheapest one. Compare its output with UCS on the same graph.*